

Learning STARK

Safe Systems Programming for Typed ML Deployment

CHAPTER 05

Functions, Structs, Enums, and Methods

Learning STARK

Chapter 5 preview: *Functions, Structs, Enums, and Methods*

Early Access manuscript - July 2026

This chapter documents a language specification under active implementation. Core v1 is a normative draft and has not yet been validated by a conforming compiler.

The design and text are original to the STARK Language Project.

Source manuscript: `book/manuscript/ch05-functions-structs-enums-methods.md`

Functions, Structs, Enums, and Methods

A production API makes data shape, ownership, construction, and behavior visible before the function body is read.

Chapters 3 and 4 established the vocabulary of values and the rules that connect paths through a program. This chapter turns those pieces into APIs. Functions define operations with explicit boundaries. Structs group facts that exist together. Enums model mutually exclusive alternatives. `impl` blocks attach construction and behavior to a type without mixing that behavior into its stored representation.

These tools are familiar, but their combination matters. A production type should make invalid construction difficult, ownership transfer visible, and common operations discoverable. Its public surface should communicate which calls only inspect a value, which calls modify it, and which calls consume it. STARK expresses those choices through function signatures and method receivers rather than through comments or naming convention alone.

The final case study builds a typed image-processing configuration. It is intentionally smaller than a complete inference service, but it demonstrates the architectural role of each construct: structs for configuration records, enums for layout and resize policy, associated functions for construction, borrowed methods for inspection, and mutable methods for controlled updates.

Specification status

The syntax and type rules in this chapter are part of the Core v1 specification. The Rust compiler scaffold is still under implementation, so runnable-looking listings describe the normative language rather than a completed toolchain. Diagnostics are illustrative unless identified by a specified code.

5.1 An API begins at a typed boundary

A function declaration names an operation and defines a boundary around it:

STARK

```
// Core v1
fn area(width: UInt32, height: UInt32) -> UInt64 {
    (width as UInt64) * (height as UInt64)
}
```

The name tells readers what the operation means. The parameter list tells callers what they must provide. The return type tells them what they receive. The body is checked against that contract independently of every call site.

Function signatures are always explicit. Parameter types are never inferred, and return types are not inferred across the boundary. A function without `->` returns `Unit`:

STARK

```
// Core v1
fn announce(message: &str) {
    println(message);
}
```

Local inference remains useful inside the body, but it cannot leak ambiguity into the API. This allows a caller, documentation generator, or separate compiler pass to understand the contract without examining the implementation.

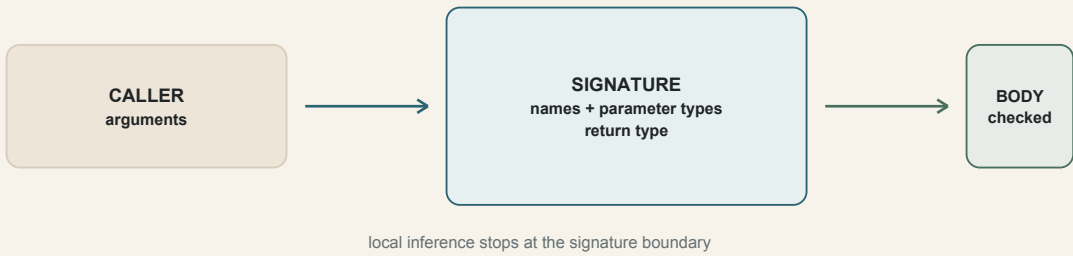


Figure 5-1. A function signature creates an explicit contract between caller and implementation.

At a call, argument count and types must match the parameters, subject only to defined coercions:

STARK

```

let pixels = area(1920u32, 1080u32);

// area(1920u32);           // Error: wrong argument count.
// area(1920u64, 1080u64); // Error: expected UInt32 arguments.

```

There is no implicit numeric narrowing or widening to rescue a mismatched call. The caller must select the intended representation explicitly.

5.2 Parameters state ownership intent

A parameter is more than an input slot. Its type describes how the callee receives access.

By-value parameters

A parameter written as `value: T` receives a value of `T`. For a non-`Copy` type, calling the function transfers ownership:

STARK

```

// Core v1
fn publish(report: String) {
    println(report.as_str());
}

let report = String::from("ready");
publish(report);
// report is no longer valid here.

```

The callee owns `report` and drops it when the call completes unless it moves the value elsewhere or returns it. By-value parameters are appropriate when the function consumes, stores, transforms, or transfers the argument.

For `Copy` values, the call copies the value instead. Passing an integer by value does not invalidate the caller's binding. The signature stays the same; the type's `Copy` property determines whether the transfer duplicates a small value or moves unique ownership.

Shared-reference parameters

A parameter `value: &T` borrows a value for inspection:

STARK

```
// Core v1
fn print_report(report: &String) {
    println(report.as_str());
}

let report = String::from("ready");
print_report(&report);
println(report.as_str()); // caller still owns it
```

The reference communicates that the function need not take ownership. Normal shared-borrow rules prevent incompatible mutation while the borrow is live.

Mutable-reference parameters

A parameter `value: &mut T` borrows exclusive mutable access:

STARK

```
// Core v1
fn increment(value: &mut UInt32) {
    *value += 1u32;
}

let mut retries = 0u32;
increment(&mut retries);
```

The caller must provide a mutable place and no conflicting live borrow may exist. The function can update the referent but does not take ownership of it.



Figure 5-2. Parameter form states whether a call transfers, shares, or exclusively borrows access.

Choose the least powerful access mode that satisfies the operation. A borrowed inspection function is easier to compose than a consuming one. A shared borrow permits more simultaneous access than a mutable borrow. A by-value parameter is clearest when ownership transfer is the point.

5.3 Return values complete the transfer

A non-`Unit` function must produce its declared type on every normal path. The trailing expression of the body supplies the value without `return`:

STARK

```
// Core v1
fn larger(a: UInt64, b: UInt64) -> UInt64 {
    if a > b { a } else { b }
}
```

An explicit `return` is useful for an early exit:

STARK

```
// Core v1
fn clamp_nonzero(value: UInt32) -> UInt32 {
    if value == 0u32 {
        return 1u32;
    }
    value
}
```

Returning an owned value moves ownership to the caller:

STARK

```
fn build_label(prefix: &str) -> String {
    let label = String::from(prefix);
    label
}
```

Returning a reference is more restricted. In Core v1, a returned reference must derive from a reference parameter on every path. A reference to a local or by-value parameter cannot escape:

STARK

```
fn choose_first(left: &String, right: &String) -> &String {
    if left.len() > 0u64 { left } else { right }
}

// fn invalid() -> &String {
//     let local = String::from("temporary");
//     &local // Error: reference to local cannot be returned.
// }
```

Core v1 uses a conservative shortest-input-lifetime rule rather than explicit lifetime parameters. Chapter 8 treats that rule in detail. At API-design time, the essential question is simple: if a function returns a borrow, the signature must expose the values that can own the referent.

5.4 Functions can be values, but closures are future work

Function types describe non-capturing functions:

STARK

```
fn(Int32, Int32) -> Int32
fn() -> Bool
fn(&str) // returns Unit
```

A named function can be passed where a compatible function type is required:

STARK

```
// Core v1
fn add(a: Int32, b: Int32) -> Int32 {
    a + b
}

fn apply(operation: fn(Int32, Int32) -> Int32,
         left: Int32,
         right: Int32) -> Int32 {
    operation(left, right)
}

let total = apply(add, 3, 4);
```

Capturing closures are a future extension. A Core v1 API should not depend on anonymous behavior that closes over local state. Named functions and explicit state parameters keep the data flow visible.

Function types include parameter and return types but not parameter names. Two functions with different internal names but identical type lists share a function type.

5.5 Structs model facts that coexist

A struct defines a product type: every value contains one field of every declared field type.

STARK

```
// Core v1
struct ImageSize {
    width: UInt32,
    height: UInt32
}
```

Constructing a value names each field:

STARK

```
let size = ImageSize {
    width: 1920u32,
    height: 1080u32
};
```

Field names make meaning independent of position. `width` and `height` have the same type, but reversing them would still change the domain meaning. A tuple could not make that distinction visible at access sites.

When a local binding has the same name as a field, field-init shorthand avoids repetition:

STARK

```
fn image_size(width: UInt32, height: UInt32) -> ImageSize {  
    ImageSize { width, height }  
}
```

Field access uses dot syntax:

STARK

```
let width = size.width;  
let height = size.height;
```

The binding's mutability controls field assignment:

STARK

```
let mut size = ImageSize { width: 640u32, height: 480u32 };  
size.width = 1280u32;
```

A public field may be declared with `pub`; fields without `pub` follow the module visibility rules. Public fields are convenient for transparent records. Private representation combined with methods gives the defining module more control over invariants. Chapter 11 develops the module boundary itself.

Reference fields are restricted in Core v1

User struct and enum declarations may not write `&T` or `&mut T` directly as field types. The simplified lifetime model cannot express arbitrary stored borrows:

STARK

```
// Not permitted in Core v1:  
// struct View {  
//     text: &str  
// }
```

A generic standard-library type may be instantiated with a reference, such as `Option<&T>`, producing a borrow-carrying value subject to conservative rules. The distinction keeps ordinary user-defined records lifetime-free while retaining essential borrowed library results.

Construction is checked field by field

A struct literal is a complete value, not a gradually populated object. Each declared field must receive a compatible expression. This makes construction a natural place for the compiler to catch representation mistakes: a misspelled field, a missing field, or a value of the wrong type cannot silently create a partial record.

Struct-literal syntax has one parsing restriction. A struct literal may not be the outermost expression in an `if` or `while` condition, a `match` scrutinee, or a `for` iterable. Braces already delimit the following control-flow block, so the restriction keeps parsing unambiguous. Parentheses make the intended boundary explicit:

STARK

```
if (ImageSize { width: 1u32, height: 1u32 }).width > 0u32 {
    println("positive width");
}
```

In ordinary code, constructing a temporary merely to inspect one field is rarely the clearest design. The example demonstrates the grammar rule; a named binding or an associated validation function usually communicates intent better.

5.6 Enums model facts that are alternatives

An enum value contains exactly one declared variant. Variants may carry no payload, positional payloads, or named fields:

STARK

```
// Core v1
enum ResizePolicy {
    None,
    Exact(UInt32, UInt32),
    Fit { max_width: UInt32, max_height: UInt32 }
}
```

The variants represent distinct states rather than partially populated fields. A struct with `should_resize`, optional width, optional height, and a mode code could admit contradictory combinations. The enum makes each valid shape explicit.

STARK

```
let unchanged = ResizePolicy::None;
let fixed = ResizePolicy::Exact(224u32, 224u32);
let bounded = ResizePolicy::Fit {
    max_width: 1024u32,
    max_height: 1024u32
};
```

Pattern matching extracts the payload and requires complete handling, as Chapter 4 showed. Structs answer “which fields exist together?” Enums answer “which one of these forms exists now?” Production domain models often combine them: a struct holds stable configuration while an enum field captures a policy choice.

Enums may have inherent `impl` blocks too. A method can match on `self` to provide behavior shared by every variant:

STARK

```
impl ResizePolicy {
    fn changes_size(&self) -> Bool {
        match self {
            ResizePolicy::None => false,
            ResizePolicy::Exact(_, _) => true,
            ResizePolicy::Fit { max_width: _, max_height: _ } => true
        }
    }
}
```

This keeps the classification rule near the variants while leaving the stored alternatives unchanged. The method still has to be exhaustive, so a new policy variant prompts a review of the behavior.

STRUCT / PRODUCT

layout

resize

normalize

ENUM / SUM

None

Exact(...)

Fit { ... }

Figure 5-3. Struct fields coexist; enum variants are mutually exclusive alternatives.

5.7 `impl` blocks attach behavior to a type

An inherent implementation block contains functions associated with a type:

STARK

```
// Core v1
impl ImageSize {
    fn square(edge: UInt32) -> ImageSize {
        ImageSize { width: edge, height: edge }
    }

    fn area(&self) -> UInt64 {
        (self.width as UInt64) * (self.height as UInt64)
    }
}
```

`square` has no receiver, so it is an associated function. Call it through the type:

STARK

```
let thumbnail = ImageSize::square(256u32);
```

`area` has `&self`, so it is a method. Call it through a value:

STARK

```
let pixels = thumbnail.area();
```

The dot-call is not dynamic magic. A method is an ordinary function whose first parameter is a receiver. The receiver syntax communicates access mode and lets the compiler insert defined auto-borrows.

Constructors are ordinary associated functions

Core v1 has no privileged constructor syntax. Names such as `new`, `from_parts`, `square`, or `default_config` are API conventions. Their behavior comes entirely from their declared parameters, return type, and body.

That simplicity is useful. A type may expose several construction paths without an overloaded language mechanism:

STARK

```
impl ImageSize {
    fn new(width: UInt32, height: UInt32) -> Self {
        Self { width, height }
    }

    fn square(edge: UInt32) -> Self {
        Self { width: edge, height: edge }
    }
}
```

An associated constructor can validate, normalize, or select a representation before returning. If construction can fail, its return type should say so using `Option<Self>` or `Result<Self, E>` rather than returning a sentinel value. Chapter 6 develops that design in full.

Three receiver modes

Core v1 supports:

- `self`, consuming the receiver by value;
- `&self`, borrowing it for shared access; and
- `&mut self`, borrowing it for exclusive mutable access.

STARK

```
impl ImageSize {
    fn width(&self) -> UInt32 {
        self.width
    }

    fn scale_by(&mut self, factor: UInt32) {
        self.width *= factor;
        self.height *= factor;
    }

    fn into_pair(self) -> (UInt32, UInt32) {
        (self.width, self.height)
    }
}
```

`width` observes without mutation. `scale_by` requires a mutable place and changes the existing value. `into_pair` consumes the structure and returns another representation.



Figure 5-4. The receiver declares whether a method consumes, observes, or mutates its value.

Receiver choice is part of the public contract. Changing `&self` to `self` can invalidate caller code by introducing a move. Changing `&self` to `&mut self` reduces the contexts in which the method can be called. Treat receiver changes as API changes, not implementation details.

5.8 `Self` keeps implementations aligned

Within an `impl` or trait block, `Self` denotes the implementing type:

STARK

```
impl ImageSize {
    fn new(width: UInt32, height: UInt32) -> Self {
        Self { width, height }
    }

    fn same_shape(&self, other: &Self) -> Bool {
        self.width == other.width && self.height == other.height
    }
}
```

Using `Self` expresses that construction and comparison stay tied to whichever type the implementation targets. It avoids repeating the type name and becomes especially valuable in generic and trait implementations.

`Self` is valid only inside a trait or `impl` block and only as the first path segment. As a complete type it means the implementing type; a longer path such as `Self::Item` refers to an associated item.

Do not confuse `Self` with `self`. Capitalized `Self` is a type-level name. Lowercase `self` is the receiver value available inside a method.

5.9 Method calls use controlled conveniences

Given `receiver.method(arguments)`, method resolution follows defined steps. The compiler first prefers an inherent method. If none applies, it considers methods from implemented traits that are in scope. Ambiguous trait methods require an explicit fully qualified call.

For receiver matching, the compiler tries by-value access, a shared auto-borrow, and then a mutable auto-borrow. A mutable auto-borrow requires a mutable place. It may also auto-dereference nested references until a candidate applies.

STARK

```
let size = ImageSize::new(640u32, 480u32);
let pixels = size.area(); // compiler can pass &size to area(&self)
```

The call remains concise, but the declaration retains the ownership truth. Reading `fn area(&self)` tells you the method does not consume the value.

Trait methods can be invoked in fully qualified form, passing the receiver explicitly:

STARK

```
let text = Display::fmt(&value);
```

That form resolves ambiguity and exposes the ordinary-function model underneath dot syntax. Traits and their coherence rules are the focus of Chapter 9; for now, remember that inherent methods are the type's direct behavior surface, while trait methods connect it to shared abstractions.

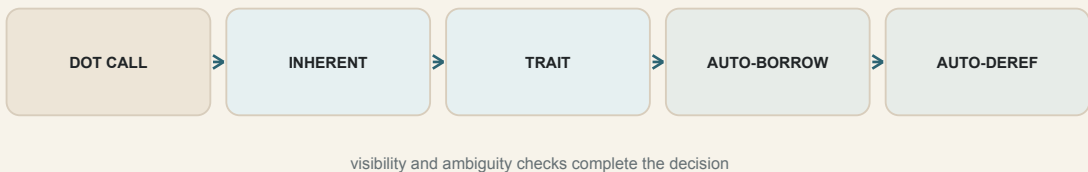


Figure 5-5. Dot-call resolution applies a small, defined search rather than arbitrary dispatch.

5.10 Separate representation from behavior

Putting a function in an `impl` block does not store code in each value. The fields still define the representation. The implementation defines operations associated with that representation.

This separation supports several useful designs:

- **Transparent data:** public fields and a small set of convenience methods.
- **Invariant-preserving type:** private fields, associated constructors, and methods that maintain valid state.
- **Domain value:** immutable-looking operations that borrow `&self` and return new values.
- **Stateful resource:** mutation through `&mut self` and deliberate consumption through `self`.

Avoid adding a method merely because it mentions the type. If an operation conceptually belongs to a different service, or treats several types symmetrically, a free function may be clearer. Conversely, constructors, invariant checks, and transformations centered on one receiver are discoverable and expressive as associated functions or methods.

The most maintainable APIs make illegal or surprising operations visually distinct. `config.validate()` promises inspection. `config.normalize()` might reasonably mutate or return a new value, so its signature must settle the question. `config.into_runtime()` conventionally signals consumption, but the `self` receiver is the enforceable fact.

5.11 Case study: a typed image configuration

An image pipeline needs more than width and height. It must know channel layout, resizing policy, and whether normalization is enabled. We can model these choices without booleans and sentinel dimensions leaking through the API.

STARK

```
// Core v1
enum PixelLayout {
    Gray,
    Rgb,
    Bgr,
    Rgba
}

enum ResizePolicy {
    None,
    Exact { width: UInt32, height: UInt32 },
    Fit { max_width: UInt32, max_height: UInt32 }
}

struct ImageConfig {
    layout: PixelLayout,
    resize: ResizePolicy,
    normalize: Bool
}
```

Each enum states a closed choice. The struct states that every configuration has one layout, one resize policy, and one normalization decision.

An inherent implementation supplies construction and queries:

STARK

```
// Core v1
impl ImageConfig {
    fn new(layout: PixelLayout) -> Self {
        Self {
            layout,
            resize: ResizePolicy::None,
            normalize: false
        }
    }

    fn with_exact_resize(&mut self,
                        width: UInt32,
                        height: UInt32) {
        self.resize = ResizePolicy::Exact { width, height };
    }

    fn enable_normalization(&mut self) {
        self.normalize = true;
    }

    fn channel_count(&self) -> UInt32 {
        match self.layout {
            PixelLayout::Gray => 1u32,
            PixelLayout::Rgb => 3u32,
            PixelLayout::Bgr => 3u32,
            PixelLayout::Rgba => 4u32
        }
    }
}
```

`new` is an associated function because no configuration exists yet. It provides a predictable default policy without exposing a partially initialized value. The two update methods require `&mut self`, making mutation explicit. `channel_count` borrows shared access and uses an exhaustive match.

Validation can remain an inspection method:

STARK

```
// Core v1
impl ImageConfig {
    fn dimensions_are_valid(&self) -> Bool {
        match self.resize {
            ResizePolicy::None => true,
            ResizePolicy::Exact { width, height } => {
                width > 0u32 && height > 0u32
            },
            ResizePolicy::Fit { max_width, max_height } => {
                max_width > 0u32 && max_height > 0u32
            }
        }
    }
}
```

The receiver is `&self` because validation does not need to modify or consume the configuration. Every resize variant produces `Bool`, so the match and method do as well.

A caller sees the access story at each line:

STARK

```
// Core v1
fn prepare_config() -> ImageConfig {
    let mut config = ImageConfig::new(PixelLayout::Rgb);
    config.with_exact_resize(224u32, 224u32);
    config.enable_normalization();

    if !config.dimensions_are_valid() {
        panic("invalid resize dimensions");
    }

    config
}
```

The local binding is mutable because two methods require mutable receivers. Validation only borrows. The final expression moves the completed configuration to the caller.

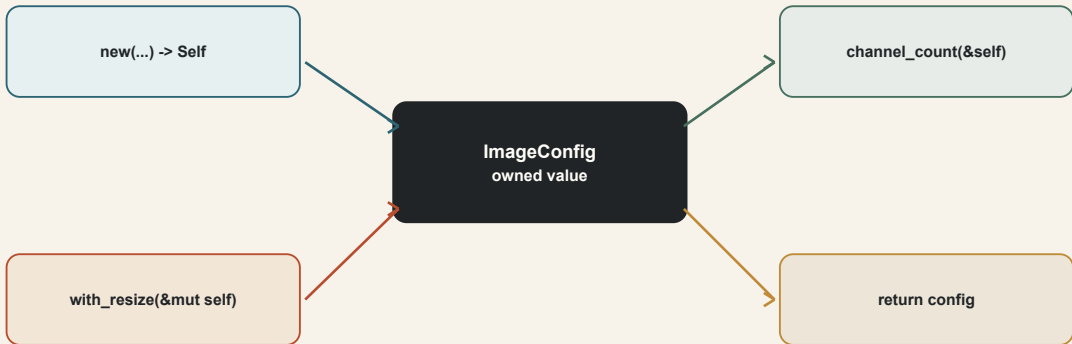


Figure 5-6. The image configuration API separates construction, mutation, inspection, and transfer.

This first version deliberately returns `Bool` from validation. A production API usually needs to explain which invariant failed; Chapter 6 will replace the Boolean with a typed `Result` and an error enum. The data model need not change to improve the failure contract.

Review the invariants at construction time

The mutable builder-like surface is readable, but it permits a temporarily invalid exact resize such as zero by zero until validation runs. Alternatives include:

- accept all required settings in an associated constructor and reject invalid input there;
- return a new configuration from each transformation rather than mutating;
- introduce a validated dimension type that cannot contain zero; or
- keep mutation internal to a module and expose only checked operations.

The right choice depends on error handling, call-site ergonomics, and whether intermediate states may escape. Types provide the construction kit; API design decides where the invariant becomes enforceable.

5.12 A practical API review checklist

Before publishing a type and its functions, review the surface systematically.

- 1. Name the domain concept.** A struct or enum name should describe meaning, not storage mechanics.
- 2. Choose product versus sum.** Fields coexist in a struct; variants exclude one another in an enum.
- 3. Make signatures complete.** Parameter and return types must communicate the entire cross-function contract.

- 4. Choose ownership deliberately.** Use `T` to consume, `&T` to inspect, and `&mut T` to modify in place.
- 5. Control construction.** Use associated functions when raw field construction would permit invalid or confusing values.
- 6. Select the receiver from behavior.** Observation uses `&self`; mutation uses `&mut self`; conversion or transfer may use `self`.
- 7. Keep return paths honest.** Every normal path must produce the declared type; returned borrows must derive from reference parameters.
- 8. Prefer explicit variants to flag combinations.** Enums make alternatives visible and exhaustively matchable.
- 9. Treat convenience as semantic.** Auto-borrowing shortens calls but must not obscure the declared receiver contract.
- 10. Plan for evolution.** Public fields, wildcard matches, and consuming receivers each affect how safely an API can change.

Good API design is not achieved by moving every free function into an `impl`. It comes from aligning names, data representation, ownership, and behavior so the easiest call is also the correct call.

Chapter summary

- Function boundaries have explicit parameter and return types; return types are never inferred.
- A function without `->` returns `Unit`; every normal path of a value-returning function must satisfy the declared type.
- By-value parameters consume non-`Copy` values, shared references inspect, and mutable references permit exclusive modification.
- Owned return values move to the caller; returned references must derive from reference parameters.
- Core v1 function types represent named, non-capturing functions; capturing closures are future work.
- Structs model fields that coexist, while enums model mutually exclusive variants.
- User-defined struct and enum fields cannot directly declare reference types in Core v1.
- Associated functions have no receiver and are called through the type.
- Methods declare `self`, `&self`, or `&mut self` to state consumption, inspection, or mutation.
- `Self` denotes the implementing type; lowercase `self` is the receiver value.
- Method calls use defined inherent-method priority, auto-borrowing, and auto-dereferencing rules.
- A well-designed API separates stored representation from behavior and makes invariants visible at construction boundaries.

Exercises

1. Write `fn perimeter(width: UInt32, height: UInt32) -> UInt64` without implicit numeric conversion.
2. For a `String` parameter, explain the caller-visible difference among `String`, `&String`, and `&mut String`.
3. Write a function that returns one of two input `&String` references. Why can it not return a reference to a local `String`?
4. Define a compatible function for the type `fn(Int32, Int32) -> Bool`, then pass it to another function.
5. Model a network endpoint as a struct with host and port. When would public fields be appropriate?
6. Replace a record containing `kind`, optional `width`, and optional `height` with an enum whose variants represent valid shapes.
7. Add `is_square(&self) -> Bool`, `double(&mut self)`, and `into_area(self) -> UInt64` to `ImageSize`. Explain each receiver.

8. Rewrite an `impl` to use `Self` consistently in its constructor return type and same-type parameters.
9. Explain why calling an `&self` method usually does not require writing `&value` at the call site.
10. Extend `PixelLayout` with `Cmyk`. Which code must change, and how does exhaustive matching help?
11. Redesign `ImageConfig` so invalid zero dimensions cannot be installed by a public mutation method.
12. Review an API you know: identify one operation that should consume, one that should borrow, and one that should be an associated function.

Source notes

1. Function signatures, receivers, structs, enums, implementation blocks, field initialization, and `Self` path rules: [STARKLANG/docs/spec/02-Syntax-Grammar.md](#).
2. Struct and enum restrictions, function types, local inference boundaries, function calls, and method resolution: [STARKLANG/docs/spec/03-Type-System.md](#).
3. Call compatibility, return-path analysis, and reference-return checking: [STARKLANG/docs/spec/04-Semantic-Analysis.md](#).
4. Ownership transfer through parameters and returns, partial moves, and automatic cleanup: [STARKLANG/docs/spec/05-Memory-Model.md](#).
5. Visibility and package-boundary behavior introduced here are developed normatively in [STARKLANG/docs/spec/07-Modules-and-Packages.md](#).